

Bloque A — Especificación Técnica

Backend y Motor de Escaneo de UYW4Q

1. Stack tecnológico

Componente	Elección	Por qué
Framework backend	FastAPI (Python)	Liviano, rápido de levantar, genera documentación automática (Swagger) que sirve directo para que el Bloque C sepa qué consumir
Base de datos + autenticación	Supabase	Ya tienes la curva de aprendizaje resuelta de Apex; incluye Postgres + auth + RPC en un solo servicio
Servidor de desarrollo	Uvicorn	Estándar para correr FastAPI
Validación de datos	Pydantic	Viene integrado con FastAPI, define los esquemas de entrada/salida sin trabajo extra
Variables de entorno	python-dotenv	Igual que ya hiciste en Apex — nunca hardcodear claves
Testing	pytest	Mismo que usaste en Apex, mantiene consistencia entre tus proyectos

2. Librerías específicas del motor de escaneo

Función	Librería sugerida	Notas
Consultas DNS (SPF, DMARC, MX, registros expuestos)	dnspython	Librería base, control total sobre las consultas
Validación de DMARC/SPF ya parseada	checkdmarc	Envuelve dnspython con parsing ya hecho para SPF/DMARC/MTA-STS — ahorra tiempo de desarrollo, revisen si su licencia permite uso comercial antes de integrarla
Certificados SSL/TLS	ssl + cryptography (librería estándar de Python + paquete cryptography)	Verifica vigencia, emisor, y configuración del certificado
Breach-check por dominio/correo	API REST de HaveIBeenPwned (de pago, ver nota de costo arriba)	Requiere hibp-api-key en el header de cada request
Cliente HTTP para llamadas externas	httpx (async)	Mejor que requests porque FastAPI es async — evita bloquear el servidor mientras esperan respuesta de HIBP

3. Estructura de carpetas sugerida

```
uyw4q-backend/  
├── app/  
│   ├── main.py           # arranque de FastAPI  
│   ├── config.py         # carga de variables de entorno  
│   └── api/  
│       ├── routes_scan.py # endpoints de escaneo  
│       └── routes_auth.py  # endpoints de autenticación (delegunderado a Supabase)  
├── core/  
│   ├── dns_checks.py      # SPF/DKIM/DMARC  
│   ├── ssl_checks.py      # certificados  
│   ├── breach_check.py    # integración HIBP  
│   └── risk_scoring.py     # combina resultados en un puntaje de riesgo  
├── models/  
│   └── schemas.py         # esquemas Pydantic (request/response)  
├── db/  
│   └── supabase_client.py # cliente de Supabase inicializado  
├── tests/  
│   ├── test_dns_checks.py  
│   ├── test_ssl_checks.py  
│   └── test_breach_check.py  
├── .env.example           # plantilla de variables (sin valores reales)  
├── requirements.txt  
└── README.md
```

4. Contrato de API (defñnanlo esta semana — bloquea al Bloque C)

POST /scan

Request:

json

```
{
  "domain": "empresa-ejemplo.pe",
  "rubro": "clinica"
}
```

(el campo `rubro` es lo que después usa el Bloque B para priorizar el riesgo según el tipo de negocio — clínica, e-commerce, academia, agencia, delivery)

Response:

```
json
{
  "scan_id": "uuid",
  "domain": "empresa-ejemplo.pe",
  "timestamp": "2026-09-14T10:00:00Z",
  "dns_findings": {
    "spf": { "present": true, "valid": true, "raw": "v=spf1 ..." },
    "dkim": { "present": false },
    "dmarc": { "present": true, "policy": "none" }
  },
  "ssl_findings": {
    "valid": true,
    "expires_in_days": 45,
    "issuer": "Let's Encrypt"
  },
  "breach_findings": {
    "emails_checked": 5,
    "emails_breached": 2,
    "breaches": [ "Interbank2024", "GenericLeak2023" ]
  },
  "risk_score": 68,
  "status": "completed"
}
```

Nota importante de seguridad y legalidad (recuérdenselo al Bloque B/D cuando integren): este endpoint solo hace **consultas pasivas** (DNS público, verificación de certificado, búsqueda en base de datos de filtraciones ya conocidas). **Nunca** escaneen puertos activos ni intenten acceder a sistemas del dominio consultado sin autorización — eso es lo que ya acordamos que evitaríamos por el riesgo legal bajo la Ley de Delitos Informáticos.

`GET /scan/{scan_id}`

Devuelve el resultado de un escaneo ya realizado (para que el frontend pueda consultar el estado si el escaneo toma unos segundos).

`POST /auth/signup` y `POST /auth/login`

Delegados directamente a Supabase Auth — no hace falta reinventar esto, usen el SDK de Supabase tal como en Apex.

5. Orden de construcción recomendado (dentro del Bloque A)

- 1. **Semana 1:** Estructura del proyecto + `(dns_checks.py)` (SPF/DMARC) funcionando de forma aislada, probado con `(pytest)` contra dominios reales conocidos (ej. su propio dominio de prueba, o dominios públicos que sepan que tienen buena/mala configuración).
- 2. **Semana 1-2:** `(ssl_checks.py)` — más simple, se puede hacer en paralelo.
- 3. **Semana 2:** Definir y congelar el contrato de API de la sección 4 — avisen al Bloque C en cuanto esté listo, no esperen a tenerlo perfecto.
- 4. **Semana 2-3:** `(breach_check.py)` — aquí es donde entra la decisión de presupuesto para la clave de HIBP; mientras deciden si pagan, pueden mockear la respuesta para no bloquear el desarrollo de las demás piezas.
- 5. **Semana 3:** `(routes_scan.py)` uniendo todo, más autenticación básica con Supabase.
- 6. **Semana 4:** Integración con el Bloque B (el LLM consume el JSON que arma `(risk_scoring.py)`) — esta es la costura más delicada, resérvense tiempo extra aquí.

6. Riesgos técnicos a vigilar (para que no los agarren de sorpresa)

- **Rate limits de HIBP:** si hacen muchas pruebas durante desarrollo, pueden agotar la cuota del plan pago rápido — usen datos simulados (mocks) para la mayoría de pruebas y solo llamen a la API real cuando necesiten validar contra datos reales.
- **Dominios sin DMARC/SPF configurado (muy común en MYPEs):** el motor tiene que manejar bien el caso de "no tiene nada configurado" sin caerse — de hecho ese es el caso más frecuente que van a encontrar y el que mejor cuenta la historia del problema en la demo.
- **Tiempo de respuesta de las consultas DNS:** algunas pueden tardar varios segundos. Consideren que el escaneo corra en segundo plano (tarea async o cola simple) y que el frontend haga polling al `(GET /scan/{scan_id})` en vez de esperar la respuesta bloqueada — mejora mucho la experiencia en la demo en vivo.